The meaning of symbolic states

Test 2 of docs/23: a higher-order slice, live borrows and holes, `Id` over them, and the step lemmas

6 November 2026 · a paper result, with a Lean check of its hardest piece and one checker probe · ochr/docs/27

Summary.

- **The meaning of a state.** A symbolic state means a function of a *future*, which gives every live borrow its final content. Under a future, a lent place means its final content, a place holding a borrow means the borrow's current content, a sealed fill means its program run with its hole read from the future, and ⊥ means nothing.
- **Ending a borrow.** Ending a borrow is fixing its final content to what it holds now. Every borrow step is a step on meanings alone. Ending a borrow, early or not, loses only information. The Lean encoding checks this on a toy machine.
- **`Id` over a live hole.** `Id` over a state with a live hole has a meaning: an observation ends every borrow in its copy, so what it compares no longer depends on the future.
- **The two obligations docs/24 named hold, and need no new rule.** A stored type never depends on a hole. Every accepted function returning a borrow has an injective backward function, by an invariant kept by every borrow step.
- **The three hard points of the first draft disappear.** `Pick`'s early end is an end, which loses information and nothing else. A3 after a drop has no second run to agree with. N3 through inert loans is meaning preservation for the fill's re-run, with the hole as one more unknown.
- **`Eq`, `J`, `match` and [Close]'s rows are monotone over ⊥. Conversion is not, and that is a closed proof of `False` on main.** Two stuck equations over moved places, from different observations, are identified by conversion. A hypothesis that is `5 = 5` at $n = 1$ then proves a goal that is `5 = 6` there. The checker accepts `Boom : False` (Section 5). The fix, specified in docs/28: nothing containing ⊥ is convertible.
- **One correction to docs/26.** A returned borrow being whole is needed for membership, not only for runtime safety.

Contents

1 What this test is . . . 3
2 What this document needs from docs/24 . . . 3
3 The meaning of a state . . . 4
3.1 Futures . . . 4
3.2 A live borrow whose loan sits in a sealed fill with a hole . . . 4
3.3 `Id` over such a state . . . 5
4 The two obligations . . . 6
4.1 A stored type never depends on a hole . . . 6
4.2 Every accepted function returning a borrow has an injective backward function . . . 6
5 Conversion is not monotone over ⊥: a closed proof of `False` . . . 7
6 The reference meaning of running a term . . . 8
7 The step lemmas . . . 9
7.1 Conversion of functions . . . 10
8 The three hard points of the first draft . . . 10
9 Monotonicity over ⊥ . . . 11
10 The historical bugs, as a check . . . 12
11 The higher-order slice . . . 12
12 The Lean check . . . 12

13 The choices 13
14 What this test does not settle 14
15 The fix on the suite 14
16 Summary for docs/18 15

1 What this test is

docs/23 proves Ochr consistent by meaning preservation. Each state the checker reaches has a meaning, as a function of what its unknowns stand for. Each step keeps it, or loses only information. Every stored value lies in its type's meaning. docs/24 (test 0) built the set model for types and for pure type-level code. This test adds two things.

- **The meaning of the state.** It covers live borrows read as functions of their final contents (the Aeneas reading), loans inside sealed fills, holes, inert loans, `⊥` as no information, and `Id` inside types.
- **The step lemmas for a higher-order slice.** The slice has function parameters ranging over every set function, a closure built in the run and called later, conversion of functions ([Conv-fun]), and a function that takes a function returning a borrow. Each lemma is one-sided: it keeps the meaning for every assignment, or loses only information.

Hardest first. Section 3 defines the meaning of a state holding a live borrow whose loan sits in a sealed fill with a hole, and `Id`'s meaning over it. Section 4 then settles the two obligations docs/24 left: that a type never depends on a hole, and that every accepted function returning a borrow has an injective backward function. Section 5 reports what the monotonicity question turned up, before the step lemmas (Section 7), the three hard points of the first draft (Section 8), and the check against the historical bugs (Section 10).

What the step lemmas may assume. docs/26 (the census audit) found that the decisions of kind *same* rest on five lemmas. They are stated here as assumptions. Section 5 shows that the fourth is false as the checker stands.

1. **Membership** (b): every stored value's meaning lies in its stored type's meaning, under every assignment and every future. It is proved together with meaning preservation.
2. **`valType?` is sound**: the meaning of a value lies in the meaning of the type the checker reads off it.
3. **Footprint coverage**: the places `Id` observes include every place either side may write, under every assignment.
4. **Conversion soundness**: convertible values have equal meanings.
5. **Well-formedness** (c): (WF1)–(WF6) of docs/18 and (WF7). (WF1)–(WF6) say that each label has at most one borrow, loans sit only where closing off allows, borrows do not form cycles, closures capture no borrow, arguments are exclusive at call points, and bindings hold values of their types. (WF7) says `&` applies only to types in $\texttt{Type}_0$.

2 What this document needs from docs/24

> **Reminder.** docs/24 defines a meaning for every term and value of a pure slice, in ZFC with inaccessible cardinals $\kappa_0 < \ldots < \kappa_N$.
>
> - `Type`i means V_{κ_i}, `Prop` means $\{\emptyset, \{\bullet\}\}$, and err is an error value outside every type.
> - Data is read untyped: a constructor value means its tag and its fields' meanings, $(\ulcorner \texttt{C} \urcorner, e_1, \ldots, e_n)$.
> - A proof means $\bullet$, read off its declared sort.
> - A type means its *good* elements only. In particular $\Pi(x : \&T).\&U$ means the functions $d \mapsto (\mathrm{fwd}(d), \mathrm{back}(d))$ whose backward function $\mathrm{back}(d) : [\![U]\!] \to [\![T]\!]$ is injective. So docs/18′s relation R is the meaning itself, and functions that agree at every argument are equal.
> - A function type with borrow parameters means functions from the parameters' starting contents to the result and the borrowed places' final contents.
> - A type-returning function carries no final contents. A function into `Prop` means a truth value.
> - A closure means a function on its annotated domain, partial (into err) where its body goes wrong.
> - A recursive function means a function by guarded recursion on the rank of its decreasing argument: a call that does not decrease means err. So [Rec] belongs to membership, not to well-formedness.
> - `J` and `rewrite` mean their transported term, and `Eq`'s meaning does not read its type.

An *assignment* γ gives each abstract value σ of a state an element $\gamma(\sigma) \in [\![\Delta(\sigma)]\!]_\gamma$, in the order the unknowns were created. It ranges over every element, including functions no program computes.

3 The meaning of a state

3.1 Futures

A state with live borrows does not yet know what its lent places will finally hold: that depends on what the program writes through the borrows before they end. Its meaning is therefore a function of that missing information.

Definition 1 (future). A *future* φ of a state Ω gives each label ℓ of a borrow in Ω (a *live* label) a value $\varphi(\ell)$: the borrow's *final content*. A future is *typed* when $\varphi(\ell) \in [\![T_\ell]\!]_\gamma$, where $\&T_\ell$ is the declared type of ℓ's borrow.

Definition 2 (meaning of a value and of a state). Under an assignment γ and a future φ, a value v means $[\![v]\!]_{\gamma,\varphi} \in U \cup \{\text{err}, \text{none}\}$:

- a loan of a live label ℓ means $\varphi(\ell)$: a lent place means what it will finally hold;
- $\text{borrow}_\ell w$ means $[\![w]\!]_{\gamma,\varphi}$: a place holding a borrow means the borrow's *current* content, which is what reading through it sees;
- a sealed program $\lceil t \rceil$ means t run from the empty state (the meaning of running a term, Section 6), with every loan in it read from φ: a fill's hole means the final content of its borrow;
- $\bot$ means none, no information;
- an abstract value σ means $\gamma(\sigma)$, and everything else as in docs/24.

A state Ω means the function $[\![\Omega]\!]_{\gamma,\varphi}$ from positions (bindings and temporaries, frame by frame) to the meanings of their contents, together with its *shape*: which positions hold borrows, of which labels.

> *Why: the two kinds of position read the future differently on purpose. A place that has lent its content will be given back the borrow's final content, so it means that. A place holding the borrow can still read and write it, so it means what is there now. [End] connects the two: it sets the final content to the current one.*

This is the shape the Lean encoding found necessary (Section 12). "A function from the live borrows' final contents to the resolved contents", one function of one argument, is not enough. A read through a borrow sees its current content, and [End] must be expressible as fixing the future.

Lemma 3 (filling a loan is fixing the future). For a value w that does not mention ℓ, $[\![v[\ell := w]]\!]_{\gamma,\varphi} = [\![v]\!]_{\gamma,\varphi[\ell \mapsto [\![w]\!]_{\gamma,\varphi}]}$. And $[\![v]\!]_{\gamma,\varphi}$ depends on φ only at the labels whose loans occur in v.

Proof sketch. By induction on v: a loan of ℓ is the only place where $\varphi(\ell)$ is read. A sealed program reads its loans through the meaning of its run, which reads them only where they occur. Lean: `ev_subst`, `ev_congr`. □

Definition 4 (ending a borrow, on meanings). Let position r hold $\text{borrow}_\ell w$. The meaning after [End ℓ] is

$$m'(\varphi) = m(\varphi[\ell \mapsto m(\varphi)(r)]) \text{ with } r \mapsto \text{none},$$

where $m = [\![\Omega]\!]_{\gamma,\cdot}$. The final content of ℓ is fixed to what r holds now, and r is forgotten.

By Lemma 3 this is exactly what the checker's [End] does: it substitutes w for every loan of ℓ and sets r to $\bot$ (Lean: `step_end`). The borrow must be whole (no $\bot$ in w), as [End] requires. w may hold loans of other borrows, a reborrow included; they travel with it, and Lemma 3 reads them from the same future.

Definition 5 (less informative). $m' \sqsubseteq m$ when every reading of m' is a reading of m: for every future φ' there is a future φ with $m'(\varphi')(p) = m(\varphi)(p)$ wherever $m'(\varphi')(p) \neq \text{none}$.

[End] is less informative than what it ends: it narrows the futures to those in which ℓ ends with its current content, and forgets the holder (Lean: `semEnd_below`). So is forgetting a place, which is what a stuck block's over-approximated move does.

3.2 A live borrow whose loan sits in a sealed fill with a hole

This is the state docs/24 named as the hardest piece. Take a caller with a holding u, and an abstract function $\sigma_g : \Pi(x : \&\,\mathsf{Nat}).\&\,\mathsf{Nat}$:

```
let t = &a;        // a ↦ loan_ℓ,  t ↦ borrow_ℓ u
let r = σ_g(t);    // stuck: σ_g is a neutral head, so [Close] with the row for &T
```

[Close] consumes t's borrow and, for a fresh label k, leaves

- $r \mapsto \text{borrow}_k\ \lceil \text{let } c = u;\ \text{let } r = \sigma_g(\&c);\ *r \rceil$,
- $a \mapsto \lceil \text{let } c = u;\ \text{let } r = \sigma_g(\&c);\ *r := \text{loan}_k;\ c \rceil$.

The fill in a holds the hole loan_k. Its borrow is r, which lives outside the fill's run, so inside that run the loan is *inert*.

Under γ, write $G = \gamma(\sigma_g)$, an element of the meaning of $\Pi(x : \&\ \mathsf{Nat}).\&\ \mathsf{Nat}$, and $(f, b) = G(u)$. By Definition 2, under every future φ:

- r means f, the current content of the returned borrow;
- a means $b(\varphi(k))$: the fill's program runs G at u, writes the hole's final content through the returned borrow, and reads c.

Since G is good, b is injective. Nothing here is a second run, and nothing is "up to resolution": the state's meaning is two set-theoretic expressions in γ and φ.

If the program then writes `*r := v` and r ends, Definition 4 fixes $\varphi(k) := v$, and a means $b(v)$ under every future (Lean: `obs_final`). docs/18′s `PF` example asks whether writing 0 and writing 1 can leave the same content in a. They cannot, because $b(0) \neq b(1)$ by injectivity (Lean: `pf_distinct`).

`Pick`'s shape. With $\sigma_p : \Pi(x, y : \&\ \mathsf{Nat}).\&\ \mathsf{Nat}$ and two lent places a and b, [Close] leaves fills in both, sharing one hole k. Under φ, (a, b) means $b_P(\varphi(k))$, where b_P is the injective backward function of $\gamma(\sigma_p)$ into pairs (Lean: `pk2_a`, `pk2_b`).

3.3 `Id` over such a state

$\mathsf{Id}(A, t, u)$ observes t and u on private copies of the state. Each observation runs its side, ends every borrow left in its copy, and reads the result and the contents of the footprint's owners (docs/18, observation). On meanings:

Definition 6 (meaning of `Id`). At a state meaning m, the observation of t runs t from m (Section 6), then ends every live borrow (Definition 4, in any order), and reads the result and every position. $\mathsf{Id}(A, t, u)$ means $\{\bullet\}$ when the two observations agree at the result and at every position that neither side leaves moved, and $\emptyset$ otherwise. A position a side leaves moved is not compared: moving is invisible to `Id`, as docs/18 already says of its adequacy ("except places one of them leaves moved out").

Ending every borrow resolves every future. After it, the observation's values do not depend on φ, so `Id`'s meaning is a function of γ alone. The order of the ends does not matter, because the relation "a loan of ℓ occurs in the content of borrow m" is acyclic (WF3): ending them in that order fixes each final content once, and Lemma 3 makes the result independent of the order.

At `Pick`'s state, an `Id` whose sides write through r depends on the hole only through the value finally written (Lean: `pk2_id`, `pk2_id_hole`). So the statement means what it says: "after writing this, the places agree".

> *Why: an `Id` formed while a borrow is live speaks about the state as it would be if every borrow ended now. That is what the checker computes, and it is a closed statement: later writes through the borrow do not change it, because a formed type captures values (RULES P2).*

Footprint coverage reduces the comparison. Definition 6 compares every position neither side moves; the checker compares the footprint's owners only. They agree by assumption 3: a side changes a position's content, other than by moving it, only by writing it or a borrow it owns, which puts the position's owners in the footprint. Where the checker's observation finds an owner moved, its equation is stuck, which says nothing (below the reference's answer). Positions outside it are equal on both sides, so their equations are true and can be dropped. That makes the checker's computation of `Id` to a conjunction exact. Owners at a call site that an argument has several of (a hole in several fills, `Pick`) are all observed, by (O4).

4 The two obligations

4.1 A stored type never depends on a hole

Lemma 7 (types are hole-free). In every state the checker reaches, no stored type contains a loan of a live borrow: not a binding's type, not the type of an abstract value in Δ, not a generalisation record's, and not a type embedded in a value (a Π-type, a type argument a constructor value records). So a stored type means the same under every future.

Proof sketch. By induction on the run.

- **Types are formed by running code on private copies.** Π-types, closures' recorded types, annotations and [Call-type] all are. A run reads a place only after [Access] has ended every borrow whose loan occurs in what it reads, including anywhere inside a neutral at its head. So a value read into a type holds no live loan, and a fill read there has been re-run hole-free.
- **An `Id` ends every borrow of its copy before it reads** (Section 3.3).
- **Nothing later puts a loan into a type.** A loan is created only by borrowing a *place*, and a type value is not a place: a constructor value's recorded parameters are not fields. [End] substitutes for loans that occur, and none do. Substituting a refinement brings in values with no loans.
- **A generalised sealed program** is the content of the matched place after [Access] has ended the loans in its head, so its record and the new abstract value's type are hole-free.

□

The one place a type is computed over a live hole. The type of a field of an *open* dependent value is read from the current content of its earlier fields. If an index field is lent, its content is a loan. The checker does not store a type mentioning it. It checks the value on a private copy in which the lent part is a fresh abstract value of its type (D64 amended, D73), and [Repack] does the same. A fresh unknown ranges over every element of its type, which is "for every future". So this is membership quantified over futures (assumption 1), not a type depending on a hole. docs/25 case 1 is the bug that broke it: before the fix, [Repack] read the lent part as its current content. In this document's terms, it read a lent place as its current content instead of its final one.

Why: a type that depended on a hole would change meaning when the borrow ends, so a value checked against it now could leave it later. The lemma says no stored type can do that, and the one check that must quantify over holes does.

4.2 Every accepted function returning a borrow has an injective backward function

This is docs/18′s (T4), now part of membership: by C1 of docs/24, a function returning a borrow is in its type's meaning only if its backward function is injective.

Lemma 8 (holes are placed injectively). In every state the checker reaches, for every live label k and every future φ, the map $v \mapsto [\![\Omega]\!]_{\gamma,\varphi[k\mapsto v]}$, restricted to the positions whose contents hold a loan of k, is injective. Call this (Inj).

Proof sketch. By induction on the run, one case per step that moves a loan or makes one.

- **[Borrow] `&p`.** p holds the new loan, which means $\varphi(k)$: the identity. [Access] first ended every borrow whose loan was inside p's content, so no other hole moves.
- **Reads, copies, moves.** [Access] ends the loans inside what is read, so read values carry no live loan. Moving a borrow moves its holder, not its hole.
- **[Assign].** The loans on the path and in the owned part of the old content are ended first (D67), so no hole is overwritten. [Drop] of the old content ends its borrows, as below.
- **[End] of a label m** substitutes m's content into its lenders. If that content held a loan of k (a reborrow), k's hole is now inside m's lender. The map from $\varphi(k)$ to that lender is the lender's context applied to m's content, and both are injective, by (Inj) for m and for k. A composite of injective maps is injective.
- **[Close], row for `&T`.** The fills mean $b_G\big(\overline{d}\big)(\varphi(k))$, injective because the called function G is good. G is an earlier global (by the outer induction), a function parameter (by the assignment, which ranges over good functions only, C1), or a closure built earlier in the run (membership at an earlier step). [Close]'s precondition makes the arguments' contents hole-free (C8), so no older hole moves.

- **[Close], other rows.** No new hole. The lent places' loans are filled by fills holding no live loan.
- **Re-running a sealed program** keeps its meaning (Section 7), so it keeps (Inj).
- **A direct call** is the callee's run, by induction.
- **A stuck block** is an anonymous function. Each arm is a run, by induction, and the assignment selects one arm.
- **Constructor contexts.** A hole inside a constructor value is one component of a tuple: injective.

□

Corollary 9 (backward functions are injective). If a function body returning $\&T$ is accepted, its meaning's backward function is injective at every argument. The same holds for a stuck block returning a borrow.

Proof sketch. At the leaf the body returns $\mathrm{borrow}_k w$, and the frame pops. Popping drops the locals. [Drop] refuses a local that holds a live loan whose borrower is in flight (D65), so every loan of k is in the cells behind the borrow parameters when the pop succeeds. The pop ends the parameters' borrows, which carries k's hole into the cells. The backward function is $v \mapsto$ the cells' contents at $\varphi[k \mapsto v]$, which is injective by (Inj). □

Verdict. Both obligations hold, and neither needs a rule the checker lacks. They rest on rules the checker has:
- [Access] ends the loans inside what it reads, and inside a neutral head;
- D67 for assignment;
- D65, which refuses to let a borrow of a dying local escape;
- D64 amended and D73, for lent index fields;
- (C8), arguments hole-free at [Close];
- C1, function types meaning only good functions, which makes every called function's backward function injective.

5 Conversion is not monotone over ⊥: a closed proof of `False`

The monotonicity question asks whether every step that *computes* on a state gives a less informative result from a less informative input. A computation that is not monotone can turn "no information" into a definite answer that more information contradicts. `Eq` is monotone because an equation with `⊥` on a side stays stuck (D68, the fix for `MovedBothSides`). Conversion is a computation too: it decides whether two values are the same. It begins with a syntactic test, `if v == w then return true` (`conv`, `Machine.lean`), and $\bot == \bot$. So it identifies two values that each say nothing about a place, as if they said the same thing.

Finding 10 (a fifth closed proof of `False`). On main (at `10c22fd9`), the checker accepts:

```
def Same (n : Nat) (x : Nat)
    (h : Id(Unit, match n { Z => (let y = x; ()), S _ => (x := 5; ()) },
                  match n { Z => (let y = x; ()), S _ => (x := 5; ()) }))
  : Id(Unit, match n { Z => (let y = x; ()), S _ => (x := 5; ()) },
             match n { Z => (let y = x; ()), S _ => (x := 6; ()) }) := h
def Boom : False := Same(S(Z), Z, refl)
```

- **At the generic call.** n is unknown, so each side is a stuck block. The block takes x by move, because arm `Z` moves it (K1(e)). After each side, x is `⊥` in the observation. x is in the footprint, since arm `S` assigns it. So the hypothesis and the goal both compute to $\mathsf{Eq}(\mathsf{Nat}, \bot, \bot)$, stuck by D68. Conversion's syntactic test identifies them, and `Same` is accepted.
- **At `Boom`'s call.** $n = 1$, so the blocks are not stuck: each side runs arm `S` directly.
  - The hypothesis's type compares 5 with 5. It is `True`, and `refl` proves it.
  - The goal compares 5 with 6. `Eq` peels the `S`s and reaches distinct constructors, so it is `False`.

Checked on the checker. The probe was built fresh at `10c22fd9` in a scratch worktree, uncommitted (`Scratch/BotConvProbe.lean`). `Same` and `Boom` are accepted. A `reject def` copy of `Boom` reports "expected rejection, but accepted". The control without blocks, whose hypothesis is `True` and goal `False`, is rejected ("the body has type ⊤, but the goal is False").

Where the step lemma fails. Conversion soundness (assumption 4) is false: `Eq`(`Nat`, ⊥, ⊥) from the hypothesis and from the goal are convertible but mean different things. Under the assignment $n = 1$, the reference run (Section 6) does not over-approximate. The hypothesis's observation reads 5 against 5 and the goal's reads 5 against 6. Each `⊥` is "no information" about a different place at a different moment, so two of them are not "the same information".

Why `MovedBothSides`'s fix did not catch it. That fix stopped `Eq`(⊥, ⊥) from *computing* to `True`, so `refl` cannot prove it. But a stuck equation can still be proved by a hypothesis of a convertible type, and here the hypothesis is provable at the instance where it is used.

The fix is specified in `ochr/docs/28-bot-conversion.md` and is on main as D83 (`98e711ed`), approved by the user. Its audit found two more routes through the same identification, `trans` and [Rewrite]'s search for occurrences, which D83 also closes: [Rewrite] finds no occurrence of a value holding `⊥`. Its known incompleteness: a type holding `⊥` is not convertible with itself (`SelfBot`). It is the rule this section argues for: a value containing `⊥` is convertible with nothing, itself included. In `conv`, that means returning `false` when `v.hasHole || w.hasHole`, before the syntactic test; `hasHole` already looks for `⊥` anywhere inside a value.

- **Tried on the probe**, in the scratch worktree, uncommitted: `Same` is rejected ("the body of Same has type Eq(Nat, ⊥, ⊥), but the goal is Eq(Nat, ⊥, ⊥)"), `Boom` with it, and the control's verdict is unchanged.
- **On the suite**, see Section 15.
- **What it costs.** A proposition containing `⊥` can never be used, since every use passes through conversion. It could never be proved by a closed term either, since `refl` proves only `True`. So the cost is completeness on statements that assert nothing.

  Why: `⊥` is the checker's way of saying "I no longer know what is here". Two such places are not known to agree, any more than two unknown numbers are. `Eq` already refuses to compute over them; conversion must refuse to identify them for the same reason.

6 The reference meaning of running a term

The step lemmas compare each checker step with a step on meanings. To say what a whole run should mean, and so what a function means, there is one reference: the meaning of running a term, $\mathcal{D}[\![t]\!]$. It sends a state meaning to a result meaning and a new state meaning, or to err. It is defined by structural recursion on t, inside docs/24′s recursion on program position and decreasing argument, with the same rules as the machine except four:

- **Every call applies a meaning.** A call never runs a body. It applies the callee's meaning: an earlier global's, an argument's under γ, or a closure's.
  - For a borrow-free result $(r, \overline{e}) = F\left(\overline{d}\right)$, the result is r, and each lent place's final content is fixed to e_i. Its loan is filled, which by Lemma 3 fixes the future.
  - For a result $\&T$, $(f, b) = F\left(\overline{d}\right)$. The result is a new borrow k holding f, and each lent place means component i of $b(\varphi(k))$.

  Here $\overline{d}$ are the arguments' contents, which are hole-free at the call by (C8).
- **[Access] ends exactly what it must.** Before a place is read, borrowed or matched, the reference ends every live borrow whose final content the place's meaning depends on, and nothing else. The checker ends every borrow whose loan *occurs* there, which is at least that, since by Lemma 3 a meaning depends only on the loans in it.
- **A match on data chooses an arm.** It reads the tag of the scrutinee's meaning, which under γ is always a constructor. Nothing is stuck and nothing is split.
- **`Id`, `Eq` and `J`** mean as in Definition 6 and docs/24. Statements run on a copy that is discarded.

A function's meaning, for a closure or a global, is read off the reference run of its body in a fresh frame. At $\overline{d}$, start with a cell holding d_i behind each borrow parameter, run the body, and pop the frame, ending the parameters' borrows. The result and the cells' contents give $(r, \overline{e})$. For a result $\&T$ they give the forward value and back(v), the cells' contents in the future where the returned borrow ends holding v. Moves forget their place, so the reference is affine like the checker; `⊥` and none coincide.

Theorem 11 (the checker's run is below the reference run). Let the checker run t from Ω to the value v and the state Ω', inside the derivation of an accepted definition. For every assignment γ of Ω there is an extension γ' to the unknowns the run created such that $\left(\llbracket v \rrbracket_{\gamma',.}, \llbracket \Omega' \rrbracket_{\gamma',.}\right) \sqsubseteq \mathcal{D}\llbracket t \rrbracket\left(\llbracket \Omega \rrbracket_{\gamma,.}\right)$, and the reference run is not err.

Proof sketch. By induction on the checker's derivation (inside docs/18′s outer induction on definitions), one case per rule (Section 7). Each step is one of three kinds. It is *exact*: its result is the reference step applied to the meaning before. Or it is an *end* the reference does not make: it narrows the futures and forgets the holder, and is less informative (Definition 4). Or it is an *over-approximation*: a stuck block takes a place by move that the selected arm would keep, so a place is forgotten. Each reference step is monotone in Definition 5, so a less informative input gives a less informative output (Section 9). The reference run does not err: every way it could err (reading a forgotten place, using an ended borrow) the checker would hit first, since its state is less informative. □

At a leaf, the result and every borrowed cell's final content are whole ([Return]). Whole meanings are defined everywhere, so on them "less informative" is equality. The leaf's meaning is then the function's meaning at γ, and membership at the leaf, with conversion soundness, places it in the goal: docs/18′s corollary.

7 The step lemmas

Each row gives the checker's step, the step on meanings it equals, and its kind: *exact*, or *below* (less informative). "Lean" names the theorem of `Ochr/Model/State.lean` that checks it on the toy machine (Section 12).

Step	On meanings	Kind	Rests on; Lean
[Borrow] `&p`	p now means $\varphi(\ell)$; the new borrow means p's old meaning	exact	[Access] made p hole-free and whole; `step_borrow`
[Read] (move)	the result is p's meaning, which is future-free after [Access]; p means none	exact	the reference is affine too
[Copy], [Clone]	the result is p's meaning; p is unchanged	exact	—
moving a borrow	the holder moves; the hole stays with the lender	exact	—
[Assign] `p := t`	p means the new value; the old content's borrows end ([Drop])	exact, then ends	D67 (no live loan in the owned part); `step_write` for a write through a borrow, which changes the current content and leaves the lender's final content alone
[Access], [End], [Drop]	Definition 4, once per ended borrow	below	the borrow whole; `step_end`, `semEnd_below`
[Close], borrow-free row	the result is r; each lent place's final content is fixed to e_i	exact	(C8); the meaning of a sealed program is its run, the reference's call
[Close], row for $\&T$	the result is a borrow k of f; each lent place means component i of $b(\varphi(k))$	exact	(C8); `step_closeG`, `step_closeP`
[Close], η at `Unit`	the result or fill is ()	exact	membership: $F\left(\overline{d}\right)$ lies in the one-element set
[Split] on σ	under γ, $\gamma(\sigma)$ has one tag i; arm i substitutes $\sigma := \mathsf{C}_i(\overline{\sigma})$ and extends γ by the tag's fields	exact	the substitution lemma; coverage (the meaning of an inductive is a disjoint union); records and η belong to the arm (D79)
re-normalisation	a sealed program's re-run means what its program means	exact	this theorem for the re-run, with each inert loan an unknown of the re-run whose value is the future's (Section 8)

[Split-gen]	a record $n \coloneqq \sigma$ is the path condition $\gamma(\sigma) = [\![n]\!]_\gamma$	exact	records die with their arm (X3)
a call of a closure, run directly	the reference applies the closure's meaning, which is the reference run of its body in a fresh frame	below	by induction on the body's run; locality (L1)–(L5): the run touches only its frame and its arguments' lenders
a call through a function parameter	closes off at once (neutral head); [Close] with $F = \gamma(\sigma_f)$	exact	assignments range over good functions (C1)
a stuck block (not in tail position)	the arm γ selects, with the block's captures	below	captures by move forget a place (K1, K2); the block's arms are checked at return (D76)
`Eq` computes	`True` at convertible sides, a conjunction for one constructor, `False` for two, stuck otherwise or over $\bot$	exact	conversion soundness; distinct tags; tuples equal by components; `Eq` does not read its type (docs/24 C10)
`Id` computes	Definition 6, reduced to the footprint	exact	footprint coverage; observations end every borrow (Section 3.3)
`J` computes or closes off	both mean the transported term (docs/24 C9)	exact	the endpoints hold no $\bot$ (reading $\bot$ is an error)
[Conv-fun]	convertible functions have equal meanings	check	Section 7.1
conversion over $\bot$	identifies two values that say nothing	**fails**	Section 5

7.1 Conversion of functions

[Conv-fun] identifies two function values when their Π-types are convertible, their captures are convertible, and the observations of their generic calls are convertible. For a result $\&T$, the observation writes one fresh σ_w through the returned borrow and observes the owners. On meanings:

- **Every argument.** The generic call binds each parameter to a fresh unknown, and an assignment extends to those unknowns with any element of their types' meanings. So the generic observation, read under every extension, is the function's meaning at every argument.
- **The whole of it.** The observation covers the whole of that meaning: the result, the final contents of the cells, and for a borrow result the forward value and the owners after writing σ_w. Since σ_w ranges over every element, the owners give the whole backward function.
- **Equal meanings.** Convertible observations have equal meanings, by conversion soundness for the values observed, so the two functions agree at every argument. By docs/24′s C1 the meaning of a function type holds only good functions, so agreeing at every argument means equal (Lean: `pi_ext`, `brwRet_ext` in `Slice.lean`).

With pairs instead of good sets this step would fail (docs/24, `Paired.differ`). Over $\bot$ it would need the fix of Section 5. A function's observation can hold $\bot$ only in a cell, and a cell must be whole at return, so in practice it never does.

8 The three hard points of the first draft

docs/18′s superseded route compared the checker's run with a run at each concrete instance. Three of its open risks were about that comparison.

`Pick`'s early end: it disappears. In `let r = Pick(n, &a, &b); let z = b; …`, at an unknown n, the hole of the returned borrow sits in the fills of both a and b. So reading b ends r, while at $n = 0$ a concrete run would not. Here there is no concrete run.

- **An end and nothing else.** Reading b is an [End] of r's borrow: it fixes the hole to r's current content and forgets r. That loses information (Lean: `pk3_step`, `pk3_below`).

- **What the reference does instead.** Under an assignment where `Pick` returned a's borrow, b's meaning does not depend on the hole, because the backward function leaves b alone. So the reference reads b without ending r (Lean: `pk2_b_at_rhoX`). What the early end lost is r, which the checker refuses to use afterwards.
- **The checker can be the more precise one.** When `Pick` runs directly on a known n, the checker's own state leaves b with no loan. The reference, which ends only what a meaning depends on, agrees.

The point moves into Definition 2 as an ordinary fact: a fill is a function of its hole.

A3 after a drop, and early ends with pairs: they disappear. Clause (A3) of the superseded route said the symbolic run had ended at least the borrows the instance run had. Its risk was a shallow early end leaving a reborrow alive on one run and not the other. Here an early end, at [Access] or [Drop], is Definition 4 whatever the shape of the ended borrow's content.

- **Reborrows.** If the content holds a loan of a reborrow, or a pair with one in either component, the loan travels with it into the lender, and Lemma 3 reads it from the same future.
- **No second borrow set.** There is no other run whose live borrows must agree. The only requirement is that the checker does not use a borrow it ended, which is its own error rule.

N3, resolution commuting with refinement through inert loans: it disappears. A refinement substitutes a constructor value for σ and re-runs the sealed programs it reaches. A fill's re-run treats its inert hole as an abstract value: [Access] does not end it, [Read] copies it, [Drop] ignores it. So the re-run is a checker run with the hole as one more unknown. Theorem 11 for that run, under the assignment extended by the hole's value in the future, says the re-normalised fill means the same function of its hole. Resolution is reading the meaning at the resolving future, refinement is a substitution that keeps the meaning, and two ways of evaluating the same function commute.

Why: each of the three was a statement about how two runs correspond, and each comes back here as either a definition (a fill is a function of its hole) or a single step that loses information.

9 Monotonicity over ⊥

Over-approximating steps leave `⊥`, or end a borrow, where the reference keeps the information. They are harmless if every step that *computes* on a state is monotone: from a less informative input it gives a less informative output, or it fails. The four the question names, and conversion:

- **`Eq`: monotone.** With `⊥` anywhere in a side (`hasHole`), it stays stuck, which says nothing (D68). Otherwise it decides from conversion, which needs no `⊥` (see below), and from constructor tags, which `⊥` in a field cannot change.
- **`J`: monotone.** Both of its outcomes, computing to t and closing off as a block whose arm is t, mean t (docs/24 C9). Its endpoints are run on private copies, and reading `⊥` is an error, so they never hold it.
- **`match`: monotone.** A match on `⊥` is an error. A match on a constructor value reads only its tag, which `⊥` inside a field does not change. A match on a neutral is stuck, and closes off.
- **[Close]'s rows: monotone.** The row (borrow, data, `()`) is read from declared types (D48, D80). No value is consulted, so `⊥` cannot change it.
- **Conversion: not monotone, and that is a closed proof of `False` (Section 5).** `⊥ == ⊥` identifies two places about which nothing is known. With the fix, conversion is monotone: a value with `⊥` is convertible with nothing.

What `⊥` inside a borrow's content means (RULES D53: moving out through a borrow, allowed when the place is refilled before the borrow ends). The borrow's current content is then partly none, and so is any read of it, which the checker refuses (reading a value holding `⊥` is an error). [End] requires the content whole, so a future is never fixed to a partial value. A refill is a write ([Assign] through the borrow), which makes the content whole again. Nothing here needs more than monotonicity.

10 The historical bugs, as a check

docs/25 read each historical closed proof of `False` as a step lemma that fails before the fix. With Definition 2 the readings sharpen.

Bug	Step whose lemma fails, before the fix	In this document's terms
D73 `BoomI2`	[Repack]'s accepting direction: membership for every future	it read a lent place as its current content; Definition 2 reads it as its final content, $\varphi(\ell)$
D75 `BoomJ`	membership at a `J` block, the producer	the block's sealed result means a value outside its stored type at some γ
D78 `BoomCapSplit`	the guard of a recursive function	the guard fires and the meaning is err (docs/24); membership fails
D79 EtaBoom	[Split]'s coverage	no assignment exists in the sibling arm, which needs an element of $\emptyset$
fuzz 11014 `MovedBothSides`	`Eq`'s monotonicity over `⊥`	`⊥ = ⊥` computed to `True`
new: `Same`, `Boom`	conversion's monotonicity over `⊥`	`⊥ ≡ ⊥` by the syntactic test (Section 5)

Each lemma holds after its fix. The new one is the conversion counterpart of `MovedBothSides`. That fix closed `Eq`, and the same reading of `⊥` (no information) closes conversion.

11 The higher-order slice

- **Function parameters range over every set function.** A parameter $\sigma_f : \Pi(x : \&\ \mathsf{Nat}).\&\ \mathsf{Nat}$ means $\gamma(\sigma_f)$, any good function, the non-computable ones included. A call through it has a neutral head, so it closes off at once, and [Close] is exact with $F = \gamma(\sigma_f)$ (Lean: `step_closeG` with `ρ.g` arbitrary).
- **A closure built in the run and called later.** Forming a closure reads its captures: [Access] ends their loans, and captures are never borrows (WF4). So the closure's meaning is fixed when it is formed and is independent of the future. It is the function on its annotated domain given by its body's reference run (docs/24 C3). A later call, direct or closed off, applies that function: a direct call is below it (Theorem 11), and a closed-off call is exact. A recursive closure that calls an enclosing function by name uses that function's guarded restriction (docs/24), so a call that does not decrease means err, which [Rec] rules out.
- **[Conv-fun]** is sound, as Section 7.1 shows.
- **A function that takes a function returning a borrow:**

```
def UseG (g : Π(x : &Nat). &Nat) (a : Nat) : Nat := (let r = g(&a); *r := 5; clone(a))
```

  Under γ, with $(f, b) = \gamma(g)(\gamma(a))$:
  - `g(&a)` closes off: r means f, and a means $b(\varphi(k))$.
  - `*r := 5` sets r's current content to 5.
  - `clone(a)` reads a, whose fill holds the hole, so [Access] ends r and fixes $\varphi(k)$ to 5. The result is $b(5)$.

  `UseG` therefore means $(G, n) \mapsto b_G(n)(5)$, an element of $\prod_{G \in \mathrm{Good}} \hat{\mathbb{N}}$, where Good is the meaning of $\Pi(x :\ \&\ \mathsf{Nat}).\&\ \mathsf{Nat}$ (Lean: `obs_final`; docs/24's `I_takes_brwRet` gives the type's meaning). By Corollary 9, two different writes through r give different results, which is what `PF` relies on.

12 The Lean check

Two standalone files, each importing Lean core only, not imported by `Ochr.lean`, and checked with `lake env lean` from `ochr/core/lean`. Both build with no `sorry`; their axioms are `propext` and `Quot.sound`. They encode a toy symbolic machine whose values are numerals, abstract numerals, `⊥`, borrows, loans, and the sealed programs

[Close] makes for opaque functions. Under an assignment, each opaque function is an element of its restricted function type (docs/24′s encoding), so its backward function is injective by construction.

`Ochr/Model/State.lean` (commits `d8918714`, `10c22fd9`).

- **The meaning of a state is a function of a future.** A lent variable means its final content, a variable holding a borrow its current content, a sealed program its call's meaning, and `⊥` none. This is Definition 2; the Lean check is what found that this shape is needed.
- **Filling a loan fixes the future** (`ev_subst`, with no side condition). A value depends on the future only at its loans (`ev_congr`).
- **Each step is a step on meanings alone**: `step_borrow`, `step_write`, `step_closeG` ([Close] of $\sigma_g(\&a)$), `step_closeP` (`Pick`'s two-place shape) and `step_end`. [End] loses only information (`semEnd_below`).
- **Runs.** `obs_final`: after `t := &a; r := σ_g(t); *r := v; end r`, a means $\mathrm{back}_{\gamma(\sigma_g)}(\gamma(\sigma_0))(v)$ under every future. `pf_distinct`: `PF`'s two writes leave different contents. `pk2_id`, `pk2_id_hole`: at `Pick`'s state an `Id`-style equation depends on the hole only through the value written. `pk3_step`, `pk3_below`: `Pick`'s early end keeps the meaning and loses only r.

`Ochr/Model/State2.lean` (commits `434f5268`, `8147a3f5`).

- **[Access] derived the checker's way**: the path, then the content; an assignment's access stops at the part the place owns; a match also looks inside a neutral head. It is below (`access_below`, `endL_below`), and a read after it does not depend on the future (`read_post_live`, `read_value_indep`).
- **Ending one borrow reaches every reborrow, and shallow ends are sound** (`q4_*`, `q5_*`, `q6_*`): the question of early ends with pairs dissolves, as Section 8 argues.
- **Nested holes.** [Close] holds for any argument (`step_closeG`), a returned borrow passed to a second call included (`n4_shape`, `n6_meaning`).
- **(WF5) is about causality.** With a live loan in an argument, a current read would depend on a later write (`w0_acausal`). [Access] ends such a loan first (`w1_borrow_ends_first`). This is the reason [Close]'s precondition (C8) is needed for Definition 2 to make sense, not only for runtime safety.
- **`⊥` inside a borrow's content** means no information (`m2_*`, `m4_*`; [End] of a partial content is an error, `m2_end_error`). `Eq` over meanings is monotone in the information order (`eqSV_mono`).
- **[Drop] is below** (`drop_below`, `dropLoop_below`). An in-flight borrower is an error (`drop_in_flight`).
- **Matches.** A match on a fill ends its borrow (`match_fill_ends`); a match on a constructor keeps the others (`match_ctor_keeps`).
- **Meanings carry `⊥` as a value**, and `Below` is the information order (`SV.le`, `Below.trans`).

Where the paper and the encoding differ.

- **Futures.** Lean's futures give every label a numeral. The paper's give a value in the borrow's content type (typed futures), which membership needs (Definition 1).
- **What is checked on runs only.** Lean checks the step lemmas at field paths on the runs it encodes. The paper argues them in general (Section 7).
- **(WF2).** Lean assumes its preservation, which needs (WF3). The paper assumes it too (assumption 5).
- **The reference run** (Section 6) is the paper's alone. The Lean files compare each symbolic step with a step on meanings, which is the exact part of Theorem 11.

13 The choices

- **D1. A state means a function of a future**, the final content of every live borrow. A lent place reads the future; a borrow's holder reads its current content (Definition 2).
- **D2. Futures are typed** for membership: each final content lies in its borrow's content type.
- **D3. `⊥` means no information**, and the reference run is affine: a move forgets its place. docs/18 already chose this, and docs/26 forced it.

- **D4. The reference run applies meanings at every call, and ends only what a read depends on** (Section 6). It is the one standard the checker's run is compared with, and it is defined from meanings, never from the symbolic representation.
- **D5. `Id` compares every position neither side moves, after ending every borrow** (Definition 6). The footprint is the checker's way of computing it, and a stuck equation over a moved owner is below the reference's answer.
- **D6. An over-approximated move forgets the place.** So it is a step below the reference, not a step to a different value.
- **D7. Conversion should identify nothing containing `⊥`.** This is a rule change for the checker, specified in docs/28 (Section 5).

14 What this test does not settle

The four hardest remaining points the coordinator named:

- **`⊥` inside a borrow's content.** Covered on paper (Section 9) and in Lean (`State2.lean`).
- **Reborrows, and a returned borrow passed into a second call** (nested holes). Covered on paper: Lemma 3 reads nested loans from one future, and [Close]'s arguments are hole-free by (C8), so the arguments' contents never depend on the future. Covered in Lean (`n4_shape`, `n6_meaning`).
- **Early ends with pairs.** Covered on paper (Section 8) and in Lean (`q4_*` to `q6_*`).
- **[Access] deriving which borrows a read ends.** Covered on paper as a sequence of ends, each below, ending at least what the read depends on (Section 6). Lean derives it the checker's way (`access_below`, `read_value_indep`).

Not covered here:

- **The full rule set.** [Open], [Repack] and dependent fields are covered only for the lent-index point of Lemma 7. `split f in t`, `rewrite` and records are covered only as path conditions, by citation. Inductive types in general are covered only through docs/24.
- **Erased runs inside the reference.** A statement runs on a copy and is discarded. The reference does the same, and the step is exact because both discard. This is argued, not checked.
- **Footprint coverage** (assumption 3) and **`valType?` soundness** (assumption 2) are assumed, as docs/26 lists them, not proved.
- **Well-formedness preservation** (assumption 5) is assumed. Lean assumes (WF2)'s preservation too.
- **Membership at every check.** Lemma 8 and Lemma 7 are the two parts docs/24 named. The rest of membership, for instance at [Repack] and at block returns, is docs/25′s reading, not re-derived here.

A correction to docs/26. docs/26 classed "a returned borrow must be whole" (ablation 14) as needed only for runtime safety. Here it is needed for membership. A returned borrow whose content holds `⊥` has a forward value with no meaning, which lies in no type. The leaf's argument also uses wholeness to turn "less informative" into equality (Theorem 11). By contrast, "borrows passed to a closing-off call must be whole" (ablation 15) stays on the runtime side: a partial argument makes the closed-off result say nothing, which is below and harmless.

15 The fix on the suite

I tried the one-line fix in the scratch worktree (`ochre-probe-bot`, at `10c22fd9`), uncommitted. The fix itself belongs to the lane docs/28 specifies. With the fix, `lake exe tests`, which re-checks every verdict of the examples and case studies, reports **1893 of 1893 verdicts as expected** (the expected total is 1893), with no failure.

- **No existing verdict changes.** No program the suite expects to be accepted proved a proposition containing `⊥`, or converted one.
- **The probe flips.** `Same` and `Boom` are rejected, and the control is still rejected.

`lake build`'s default targets, the project's definition of correct, check the same verdicts. I did not run them separately; the lane should, after adding the twin tests.

16 Summary for docs/18

1. **The meaning of a state.** A state means a function of a *future*, the final contents of its live borrows. A lent place means its final content and a borrow's holder its current content. [End] fixes the future at the current content, and loses only information. A fill means its program with the hole read from the future. Checked in Lean (`Model/State.lean`, `Model/State2.lean`).
2. **Both obligations hold, with no new rule.** Stored types never hold a live loan, and the one type computed over a lent field quantifies over the hole (D64, D73). Every accepted borrow-returning function's backward function is injective, by an invariant that [Access], D67, D65 and (C8) keep, with called functions good by C1.
3. **`Pick`, A3 after a drop with early ends with pairs, and N3 through inert loans all disappear.** They were statements about two runs agreeing. Here they are a definition (a fill is a function of its hole), a step that loses information (an early end), and meaning preservation for a fill's re-run (the hole as one more unknown).
4. **Monotonicity.** `Eq`, `J`, `match` and [Close]'s rows are monotone over `⊥`. Conversion is not: `⊥ ≡ ⊥` gives a fifth closed proof of `False` on main (`Same`, `Boom`), fixed by docs/28 (nothing holding `⊥` is convertible).
5. **A whole returned borrow** is a membership requirement, not only a runtime one, which corrects docs/26.